

Technical Requirements Document (TRD)

Project: Retail Shelf Monitoring Using Computer Vision and Data Science

1. Purpose

This document specifies the technical implementation details of the Retail Shelf Monitoring system, including architecture, technology stack, hardware/software requirements, data formats, model specifications, and deployment approach, to guide development and evaluation.

2. System Architecture

- **Presentation Layer:** Streamlit web dashboard for image upload and result visualization.
- **Application Layer:** Python backend orchestrating preprocessing, inference, and analysis logic.
- **Model Layer:** YOLOv8n object detection model (Ultralytics) for product detection.
- **Data Layer:** SQLite database for storing detection logs, product categories, and zone configurations.
- **Storage Layer:** Local file system for uploaded images, model weights, and exported reports.

Data flow: Image Upload → Preprocessing (OpenCV) → YOLOv8n Inference → Post-processing (counting, empty-space, misplacement logic) → Results stored in SQLite → Rendered on Streamlit dashboard → Exported as CSV/image.

3. Technology Stack

Layer	Technology	Version (recommended)
Programming Language	Python	3.10+
Computer Vision	OpenCV	4.x
Object Detection	Ultralytics YOLOv8 (nano)	8.x
Data Processing	Pandas, NumPy	Latest stable
Visualization	Matplotlib, Plotly	Latest stable
Dashboard/Frontend	Streamlit	1.x
Database	SQLite	3.x
Optional LLM Runtime	Ollama (Llama 3.2 3B)	Latest stable
Version Control	Git / GitHub	-
IDE	VS Code / Jupyter / Google Colab	-

4. Hardware Requirements

Component	Minimum	Recommended
Processor	Intel i5 / equivalent	Intel i7 / Ryzen 7 or better
RAM	8 GB	16 GB
GPU	Not mandatory; CPU inference works with YOLOv8n	NVIDIA GPU (4GB+ VRAM) or Google Colab free GPU for training
Storage	5 GB free space	10+ GB free space
OS	Windows 10 / Ubuntu 20.04+ / macOS	Same

5. Software Requirements

- Python 3.10 or higher
- pip / conda package manager
- Required libraries: ultralytics, opencv-python, pandas, numpy, matplotlib, streamlit, sqlite3 (built-in)
- Optional: ollama for local LLM summary generation
- Git for version control; GitHub for repository hosting

6. Data Requirements

6.1 Input Data Format

- Image formats: JPG, PNG
- Recommended resolution: 640×640 to 1280×1280 pixels (resized internally to model input size)

6.2 Training Data Format

- Images with corresponding YOLO-format annotation files (.txt) containing class ID and normalized bounding box coordinates
- Dataset split: approximately 70% training, 20% validation, 10% testing

6.3 Output Data Format

- Detection results: JSON/CSV with fields — image_id, class, confidence, bounding_box (x, y, w, h), zone, status
- Exported report: CSV file with product counts, empty-zone flags, and misplacement flags

7. Model Specifications

Parameter	Specification
Model	YOLOv8n (nano variant)
Input size	640 × 640 pixels
Output	Bounding boxes, class labels, confidence scores
Framework	PyTorch (via Ultralytics library)
Training approach	Transfer learning from COCO-pretrained weights, fine-tuned on custom shelf dataset
Confidence threshold	Configurable (default 0.5)
IoU threshold (NMS)	Configurable (default 0.45)
Inference time (CPU)	Approximately 1–3 seconds per image on laptop-class hardware

8. Module Interfaces

Module	Input	Output
Preprocessing	Raw image file	Resized, normalized image array
Detection	Preprocessed image	List of bounding boxes + class + confidence
Counting	Detection list	Product count per category
Empty-space analysis	Detection list + shelf zone map	Empty-zone percentage per zone
Misplacement check	Detection list + zone-category map	List of misplaced items
Dashboard	All processed results	Visual display, charts, alerts, export files

9. Database Design (SQLite)

Minimal schema to support logging and configuration:

- **detections**(id, image_id, class_name, confidence, bbox_x, bbox_y, bbox_w, bbox_h, zone_id, timestamp)

- **zones**(zone_id, zone_name, expected_category)
- **reports**(report_id, image_id, total_products, empty_zones_count, misplaced_count, shelf_status, generated_at)

10. Security & Data Handling

- System runs locally; no data is transmitted to external/cloud services by default.
- Uploaded images and reports are stored on local disk only.
- No personally identifiable information (PII) is collected or processed.

11. Performance Requirements

- End-to-end processing (upload to result display) should complete within 5 seconds per image on recommended hardware.
- Dashboard should remain responsive for images up to 10 MB in size.
- Model should achieve a minimum mAP@0.5 of 0.6 on the validation set for MVP acceptance.

12. Deployment Architecture

- Local deployment: Run via `streamlit run app.py` on the development laptop.
- Optional free-tier deployment: Streamlit Community Cloud or Hugging Face Spaces for demo access.
- No containerization required for MVP; Docker packaging can be a future enhancement.

13. Development & Testing Environment

- Development: VS Code / Jupyter Notebook
- Model training/experimentation: Google Colab (free GPU tier)
- Version control: Git with GitHub repository for source code and documentation
- Testing: Manual testing of detection accuracy on validation set; unit tests for counting/analysis logic using pytest (optional)